Core Java Interview Questions

17 essential questions with explanations and code examples

1. What Is the Difference Between JDK, JRE, and JVM?

Imagine JVM as a virtual engine that runs Java programs, JRE as a ready-to-use car with the engine installed, and JDK as a complete workshop with tools, parts, and manuals for building and maintaining cars.

JVM (Java Virtual Machine)

Definition: JVM is an abstract computing machine that enables Java bytecode to be executed on different hardware platforms.

Functionality: It interprets Java bytecode into machine code or executes it using Just-In-Time (JIT) compilation, providing platform independence for Java applications.

Use Case: Suppose you have a Java application compiled into bytecode. The JVM executes this bytecode, translating it into machine-specific instructions for the underlying hardware.

JRE (Java Runtime Environment)

Definition: JRE is a runtime environment that contains JVM, libraries, and other components required to run Java applications but does not include development tools.

Functionality: It provides the runtime environment necessary for executing Java applications, including the JVM, core libraries, and additional runtime resources.

Use Case: When you want to run a pre-built Java application on your system, you need JRE installed. It provides the necessary runtime environment for the application to execute.

JDK (Java Development Kit)

Definition: JDK is a software development kit that includes JRE, development tools, libraries, and documentation necessary for developing Java applications.

Functionality: It provides everything included in JRE, along with additional tools such as compilers, debuggers, and development APIs for creating Java applications.

Use Case: If you're a Java developer, you'll use JDK to write, compile, and debug Java code. It includes everything you need for Java development, from writing code to deploying applications.

2. What Is the Difference Between Stack and Heap?

The stack and heap are two types of memory storage in the Java Virtual Machine (JVM).

The stack is used for storing method invocation and local variables. Each thread in a program has its own stack, and it operates in a last-in, first-out manner. When a method is called, a new stack frame is created, and when the method execution is completed, the stack frame is removed. Stack memory is typically faster to access than heap memory.

The heap is a shared memory pool used for dynamic memory allocation. Objects created in Java are stored in the heap. Unlike the stack, memory in the heap can be accessed randomly. Garbage collection is responsible for managing memory in the heap by reclaiming memory occupied by objects that are no longer needed.

In summary, the stack is used for storing method call information and local variables with a predictable, limited size per thread, while the heap is used for dynamic memory allocation of objects with a larger, shared memory space.

Always remember that the stack has much less memory space than the heap and when a stack is full, the JVM will throw a StackOverflowError.

3. Why is the main method in Java declared as public static void main(String[] args)?

The main method is declared this way for the following reasons:

- **public:** So the JVM can access it from outside the class.
- **static:** Allows the JVM to invoke it without creating an instance of the class.
- **void:** Indicates that the method does not return any value.
- **String[] args:** Used to accept command-line arguments when the program runs.

4. What Is the void Type and When is it preferable to Use It?

In Java, the void type is used to indicate that a method does not return any value. Using void appropriately makes the code more readable and helps communicate the intent of the method to other developers. It indicates that the method performs an action or has a side effect without returning any result.

```
// Example 1: Method with side effects
void writeToLogFile(String message) {
    // Write message to log file
}

// Example 2: Event handler method
void onButtonClick() {
    // Perform action when button is clicked
}

// Example 3: Setter method
void setName(String name) {
    this.name = name;
}

// Example 4: Utility method
void printMessage(String message) {
    System.out.println(message);
}

// Example 5: Lifecycle method
void init() {
    // Initialize resources
}
```

5. Which Access Modifiers Are Available in Java and their Purpose?

In Java, there are four access modifiers available, each serving a specific purpose:

Public (public)

Purpose: Allows unrestricted access to the member (class, method, or field) from any other class or package.

Example: A public method in a class can be accessed by any other class, even if it's in a different package.

Private (private)

Purpose: Restricts access to the member to only within the same class.

Example: A private field in a class can only be accessed and modified by methods within the same class.

Protected (protected)

Purpose: Allows access to the member within the same package and by subclasses (whether they are in the same package or different package).

Example: A protected method in a superclass can be accessed by its subclass, regardless of whether the subclass is in the same package or a different package.

Default (Package-Private)

Purpose: Restricts access to the member to only within the same package. If no access modifier is specified, it is considered default (package-private).

Example: A default (package-private) class member can be accessed by any class within the same package but not by classes outside the package.

These access modifiers help control the visibility and accessibility of classes, methods, and fields, allowing developers to enforce encapsulation, data hiding, and code organization in their Java programs.

6. Why Is String an Immutable Class?

String is immutable in Java, meaning its state cannot be changed after it's created. There are several reasons why String is designed to be immutable:

Thread Safety

Immutable objects are inherently thread-safe because their state cannot be modified once created. This eliminates the need for synchronization when working with Strings in a multi-threaded environment.

String Pool Optimization

Strings are stored in a string pool, which is a pool of unique String literals in memory. Immutable strings allow for string interning, where multiple references to the same string literal point to the same memory location. This reduces memory overhead and improves performance.

Security

Immutable strings are inherently more secure because their values cannot be changed once created. This prevents malicious code from modifying sensitive string data.

Caching and Optimization

Immutable strings can be cached and reused, leading to performance optimizations in applications. Operations like substring, concatenation, and conversion to upper/lower case can be optimized by reusing existing string instances.

Hashing and Hashcode Stability

Immutable strings have a stable hashcode, meaning their hashcode value remains constant throughout their lifetime. This property is essential for objects used in hash-based collections like HashMap and HashSet.

7. Explain Immutable nature of String with example?

```
public static void main(String[] args) {
    // Creating a string "Hello" and assigning it to str1 (stored in the string pool in the
    heap)
    String str1 = "Hello";

    // Creating another string "Hello" (refers to the same object in the string pool)
```

```
    String str3 = "Hello"; // Due to string interning, it refers to the same object in the
    string pool

    // Concatenating "World" to str1 and assigning it to str2
    String str2 = str1 + " World"; // New string created and stored in the heap

    // Converting str1 to uppercase and assigning it to str1
    str1 = str1.toUpperCase(); // New string created and stored in the heap

    // Outputting the values of str1, str2, and str3
    System.out.println("str1: " + str1); // Output: "HELLO"
    System.out.println("str2: " + str2); // Output: "Hello World"
    System.out.println("str3: " + str3); // Output: "Hello"
}
```

8. What is the concept of String Intern?

The concept of string internment in Java refers to the process of storing unique string literals in a special memory area known as the "string pool." When a string literal is encountered in Java code, the JVM checks if an equivalent string already exists in the string pool. If it does, the JVM returns a reference to the existing string object from the pool rather than creating a new object.

This concept ensures that only one copy of each distinct string literal is maintained in memory, reducing memory usage and improving performance by facilitating string sharing and reuse.

String interning can be explicitly invoked using the intern() method provided by the String class. When called on a string object, the intern() method checks if an equivalent string already exists in the string pool. If not, it adds the string to the pool and returns a reference to it. If an equivalent string already exists, it returns a reference to the existing string object from the pool.

```
public static void main(String[] args) {
    String str1 = "Hello";
    String str2 = "Hello";
    String str3 = new String("Hello").intern();

    System.out.println(str1 == str2); // Output: true (Same object reference)
    System.out.println(str1 == str3); // Output: true (Same object reference)
}
```

9. What happens when you create a String with a new keyword?

When the new keyword is used to create a string object, the JVM creates the object in the heap memory, regardless of whether an equivalent string already exists in the string pool. If the string is not found in the string pool, the JVM creates a new string object in the heap and returns a reference to it.

Here's a simplified explanation of the process:

When new keyword is used to create a string object, the JVM creates the object in the heap memory. If the string already exists in the string pool, a new object is still created in the heap, and the string is not added to the pool again.

If the string does not exist in the string pool, the JVM adds it to the pool and returns a reference to the newly created object in the heap.

This behavior ensures that each call to new String("...") creates a new string object in the heap, regardless of whether an equivalent string exists in the string pool.

This can be useful in scenarios where you explicitly want to avoid string interning and ensure that a new object is created in the heap. However, it's important to note that using new unnecessarily can lead to

unnecessary memory usage and should be done judiciously.

```
public static void main(String[] args) {
    // Creating a string object using the new keyword
    // This creates a new object in the heap memory
    String str1 = new String("Hello");

    // Creating another string object with the same value
    // This also creates a new object in the heap memory
    String str2 = new String("Hello");

    // Checking if str1 and str2 reference the same object
    // They are not the same object because each call to new creates a new object
    System.out.println("str1 == str2: " + (str1 == str2)); // Output: false

    // Creating a string object without using the new keyword
    // This checks the string pool and creates a new object only if the string is not already
    present
    String str3 = "Hello";

    // Checking if str1 and str3 reference the same object
    // They are not the same object because str3 is created in the string pool
    System.out.println("str1 == str3: " + (str1 == str3)); // Output: false
}
```

10. What is JAR vs WAR vs EAR?

JAR, WAR, and EAR are all archive file formats used in Java for packaging and deploying applications. Here's a brief overview of each:

JAR (Java ARchive)

Purpose: JAR files are used to package Java class files, resources, and metadata into a single archive file.

Contents: They typically contain compiled Java classes, libraries (JAR files), configuration files, and resources (images, properties files, etc.).

Usage: JAR files are commonly used for packaging libraries, standalone applications, and executable Java applications.

Extension: .jar

WAR (Web ARchive)

Purpose: WAR files are specifically used for packaging and deploying web applications in Java EE (Enterprise Edition) environments.

Contents: They include web components such as servlets, JSP files, HTML pages, JavaScript files, CSS stylesheets, and other resources needed for web applications.

Structure: WAR files have a defined directory structure (e.g., WEB-INF) to organize web application resources and configuration files.

Usage: WAR files are deployed to servlet containers (e.g., Apache Tomcat, Jetty) to run Java web applications.

Extension: .war

EAR (Enterprise ARchive)

Purpose: EAR files are used for packaging and deploying enterprise-level Java EE applications, which typically consist of multiple modules and components.

Contents: They can contain one or more WAR files, EJB (Enterprise JavaBeans) JAR files, application client JAR files, library JAR files, and other resources.

Structure: EAR files have a hierarchical structure, with modules organized into directories and subdirectories.

Usage: EAR files are deployed to Java EE application servers (e.g., JBoss, WebSphere, WebLogic) to run large-scale enterprise applications.

Extension: .ear

In summary, JAR files are generic archive files used for packaging Java applications and libraries. WAR files are specialized for web applications and contain web-specific resources. EAR files are used for large-scale enterprise applications containing multiple modules and components. Each type of archive serves a specific purpose in Java application development and deployment.

11. Java is strictly pass-by-value language?

Yes, Java is strictly pass-by-value language. When you pass arguments to a method in Java, the values of the arguments are copied and passed to the method parameters.

However, it's important to note that the behavior can sometimes be confusing, especially when dealing with objects. When you pass an object to a method, you are passing a reference to the object, not the object itself. This reference is also passed by value, meaning a copy of the reference is passed to the method. This can lead to the misconception that Java uses pass-by-reference, but in reality, it's still pass-by-value.

Here's a brief summary:

- For primitive data types (int, double, boolean, etc.), the value itself is passed.
- For objects, the reference to the object is passed by value. This means changes to the object's state made inside the method are reflected outside the method, but reassigning the reference inside the method does not affect the reference outside the method.

```
public class PassByValueExample {
    public static void main(String[] args) {
        int num = 10;
        System.out.println("Before calling modifyValue: num = " + num);

        // Passing a primitive type (int) by value
        modifyValue(num);

        System.out.println("After calling modifyValue: num = " + num);

        StringBuilder sb = new StringBuilder("Hello");
        System.out.println("Before calling modifyReference: sb = " + sb);

        // Passing an object (StringBuilder) by value (passing reference by value)
        modifyReference(sb);

        System.out.println("After calling modifyReference: sb = " + sb);
    }

    // Method to modify a primitive value
    public static void modifyValue(int num) {
        num += 5;
        System.out.println("Inside modifyValue: num = " + num);
    }

    // Method to modify an object reference
    public static void modifyReference(StringBuilder sb) {
        sb.append(" World");
        System.out.println("Inside modifyReference: sb = " + sb);
```

```
    }
}

// Output
// Before calling modifyValue: num = 10
// Inside modifyValue: num = 15
// After calling modifyValue: num = 10
// Before calling modifyReference: sb = Hello
// Inside modifyReference: sb = Hello World
// After calling modifyReference: sb = Hello World
```

12. Explain about static, abstract and final keywords in Java?

static

- The static keyword in Java is used to declare members (variables and methods) that belong to the class itself, rather than to instances of the class.
- Variables declared as static are called class variables or static variables. They are shared among all instances of the class and can be accessed using the class name.
- Methods declared as static are called class methods or static methods. They belong to the class itself and can be called without creating an instance of the class.
- static variables and methods can be accessed directly using the class name, without the need to create an object of the class.

abstract

- The abstract keyword in Java is used to declare abstract classes and methods.
- Abstract classes cannot be instantiated directly; they serve as templates for concrete subclasses to extend and implement.
- Abstract methods are declared without an implementation and must be implemented by concrete subclasses.
- Abstract classes may contain both abstract and concrete methods, but a class with at least one abstract method must be declared abstract.
- Abstract methods are intended to be overridden by subclasses to provide specific implementations.

final

- The final keyword in Java is used to restrict the modification of classes, methods, and variables.
- When applied to a class, the final keyword prevents the class from being subclassed (i.e., it becomes a final class).
- When applied to a method, the final keyword prevents the method from being overridden by subclasses.
- When applied to a variable, the final keyword indicates that the variable's value cannot be changed after initialization. For primitive types, it makes the value immutable, while for reference types, it makes the reference immutable (i.e., the reference cannot be changed, but the object's state can).
- final variables must be initialized either when declared or in a constructor, and they cannot be reassigned.

In summary, the static keyword is used to define members that belong to the class itself, the abstract keyword is used to declare abstract classes and methods, and the final keyword is used to restrict modification of classes, methods, and variables.

```
// StaticExample class with a static variable and method
class StaticExample {
    // Static variable
```

```
    static int staticVariable = 10;

    // Static method
    static void staticMethod() {
        System.out.println("Inside staticMethod: staticVariable = " + staticVariable);
    }
}

// AbstractExample abstract class with an abstract method
abstract class AbstractExample {
    // Abstract method (no implementation)
    abstract void abstractMethod();
}

// Concrete subclass of AbstractExample
class ConcreteExample extends AbstractExample {
    // Implementation of abstract method
    void abstractMethod() {
        System.out.println("Inside abstractMethod in ConcreteExample");
    }
}

// FinalExample class with final variable and method
class FinalExample {
    // Final variable
    final int finalVariable = 20;

    // Final method
    final void finalMethod() {
        System.out.println("Inside finalMethod: finalVariable = " + finalVariable);
    }
}

public class KeywordExample {
    public static void main(String[] args) {
        // Accessing static variable and method using class name
        System.out.println("Static variable: " + StaticExample.staticVariable);
        StaticExample.staticMethod();

        // Creating an instance of ConcreteExample and calling abstractMethod
        ConcreteExample concrete = new ConcreteExample();
        concrete.abstractMethod();

        // Creating an instance of FinalExample and accessing final variable and method
        FinalExample finalExample = new FinalExample();
        System.out.println("Final variable: " + finalExample.finalVariable);
        finalExample.finalMethod();
    }
}

/* Output
Static variable: 10
Inside staticMethod: staticVariable = 10
Inside abstractMethod in ConcreteExample
Final variable: 20
Inside finalMethod: finalVariable = 20
*/
```

13. Difference between String, StringBuffer, and StringBuilder?

The main difference between String, StringBuffer, and StringBuilder in Java lies in their mutability, thread safety, and performance characteristics:

String

- String objects are immutable, meaning their values cannot be changed after they are created.

- Once a String object is created, its value cannot be modified. Any operation that appears to modify a String actually creates a new String object with the modified value.
- String objects are thread-safe because they are immutable, making them safe for use in multi-threaded environments.
- Due to their immutability, String objects are safe for sharing across different threads without synchronization overhead.
- Use String when you need an immutable sequence of characters.

StringBuffer

- StringBuffer is a mutable sequence of characters, designed for use in multi-threaded environments.
- StringBuffer provides synchronized methods for appending, inserting, and modifying characters, making it thread-safe.
- Modifications to StringBuffer objects are performed in-place, avoiding unnecessary memory allocation and string copying.
- Use StringBuffer when you need a mutable sequence of characters in a multi-threaded environment.

StringBuilder

- StringBuilder is similar to StringBuffer in that it provides a mutable sequence of characters. However, it is not inherently thread-safe.
- Unlike StringBuffer, StringBuilder does not provide synchronized methods, making it more efficient in single-threaded scenarios.
- StringBuilder is recommended for use in single-threaded environments or when synchronization is not required.
- Use StringBuilder when you need a mutable sequence of characters in a single-threaded environment.

In summary, String is immutable and thread-safe, StringBuffer is mutable and thread-safe, and StringBuilder is mutable but not inherently thread-safe. The choice between them depends on the specific requirements of your application, such as mutability, thread safety, and performance considerations.

```
public static void main(String[] args) {
    // String Example
    String str = "Hello";
    System.out.println("Original String: " + str);

    // Concatenating a new string to the original string
    str = str + " World";
    System.out.println("Modified String (using String): " + str);

    // StringBuffer Example
    StringBuffer stringBuffer = new StringBuffer("Hello");
    System.out.println("Original StringBuffer: " + stringBuffer);

    // Appending a new string to the original StringBuffer
    stringBuffer.append(" World");
    System.out.println("Modified StringBuffer: " + stringBuffer);

    // StringBuilder Example
    StringBuilder stringBuilder = new StringBuilder("Hello");
    System.out.println("Original StringBuilder: " + stringBuilder);

    // Appending a new string to the original StringBuilder
    stringBuilder.append(" World");
    System.out.println("Modified StringBuilder: " + stringBuilder);
}
```

14. How Can We Reverse a String in Java?

The common approach to reverse a string in Java is by using either StringBuilder or StringBuffer along with their reverse() method as shown below:

```
String reversed_sBuilder = new StringBuilder("hungryCoders").reverse().toString();
System.out.println(reversed_sBuilder); // sredoCyrgnuh

String reversed_sBuffer = new StringBuffer("hungryCoders").reverse().toString();
System.out.println(reversed_sBuffer); // sredoCyrgnuh
```

15. What Is String.format() and How to Use It?

String.format() is a method in Java that allows you to create formatted strings by substituting placeholders with values.

```
String formattedString = String.format("Hello, %s! Welcome to %s.", "Friend", "HungryCoders.
    com");
System.out.println(formattedString); // Hello, Friend! Welcome to HungryCoders.com.
```

16. What Is an Enum and How to Use It?

An enum, short for "enumeration," is a special data type in Java used to define a collection of constants. It provides a way to represent a fixed set of predefined values.

```
// Defining an enum called DayOfWeek
enum DayOfWeek {
    SUNDAY, MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY
}

public class EnumExample {
    public static void main(String[] args) {
        // Using enum constants
        DayOfWeek today = DayOfWeek.MONDAY;
        System.out.println("Today is: " + today);

        // Switch statement with enum
        switch (today) {
            case MONDAY:
                System.out.println("It's the start of the week.");
                break;
            case FRIDAY:
                System.out.println("TGIF!");
                break;
            default:
                System.out.println("Just another day.");
        }

        // Iterating over enum constants
        System.out.println("Days of the week:");
        for (DayOfWeek day : DayOfWeek.values()) {
            System.out.println(day);
        }
    }
}

/* Output
Today is: MONDAY
It's the start of the week.
Days of the week:
```

```
SUNDAY
MONDAY
TUESDAY
WEDNESDAY
THURSDAY
FRIDAY
SATURDAY
*/
```

17. What Are Two Types of Casting in Java?

In Java, implicit casting (widening) is a form of upcasting, while explicit casting (narrowing) is a form of downcasting.

Upcasting (Implicit Casting)

- Upcasting involves converting an object of a subclass type to an object of its superclass type.
- It is performed automatically by the compiler when assigning a subclass object to a superclass reference.
- Upcasting is considered safe because it involves moving from a more specific type to a more general type.

Downcasting (Explicit Casting)

- Downcasting involves converting an object of a superclass type to an object of its subclass type.
- It requires an explicit cast operator and may result in a ClassCastException if the object being casted is not actually an instance of the target subclass.
- Downcasting is considered risky because it involves moving from a more general type to a more specific type, which may lead to runtime errors if not done properly.

In summary, implicit casting (upcasting) is converting from a subclass to a superclass type, while explicit casting (downcasting) is converting from a superclass to a subclass type. Upcasting is safe and implicit, while downcasting requires an explicit cast operator and may result in runtime errors if not done properly.

```
int num1 = 10;
double num2 = num1; // Implicit casting from int to double
System.out.println(num2); // 10.0

double num3 = 10.5;
int num4 = (int) num3; // Explicit casting from double to int
System.out.println(num4); // 10
```

18. Is Java statically and strongly typed? Explain in detail?

Java is both Statically and Strongly Typed.

Statically Typed: You cannot declare a variable without a type, and the type is checked at compile-time.

Strongly Typed: Type rules are strictly enforced, and incompatible types cannot mix without explicit conversions.

Statically Typed

- In Java, you must declare the type of every variable explicitly (or inferred, as with var introduced in Java 10)
- Type checking occurs during compilation. If there is a type mismatch, the compiler throws an error.
- Types are fixed after declaration and cannot change.

Being a statically typed language have lot of benefits such as:

- It prevents many runtime errors.
- It improves performance because type information is resolved during compilation.

```
int number = 10;  // Declared as int
number = "Hello"; // Compilation Error: Type mismatch
```

Strongly Typed

In Java, type conversions must follow strict rules, and incompatible types cannot be mixed without explicit conversion.

- Implicit type conversions (type coercion) are limited.
- Operations between incompatible types are not allowed.
- Java enforces strict type safety in collections (since Java 5 with generics)

Being a strongly typed language have lot of benefits such as:

- Avoids unpredictable behavior due to unintended type conversions.
- Ensures code reliability and readability.

```
int num = 10;
double result = num;  // Allowed: Implicit widening (int to double)
int value = result;   // Compilation Error: Explicit narrowing required

int x = 5;
String y = "10";
int sum = x + y; // Compilation Error: Incompatible types

List<String> list = new ArrayList<>();
list.add(42); // Compilation Error: Incompatible type
```

19. Does var keyword makes Java as a dynamically typed language?

Java introduced the var keyword in Java 10, which allows you to declare local variables without explicitly specifying their type. However, this does not mean Java has become a dynamically typed language. It is still statically and strongly typed, even with var.

Basically var is a reserved type name introduced to simplify local variable declarations. The compiler infers the type of the variable from the value assigned to it during initialization.

With var, the type is determined at compile-time, not at runtime. The compiler infers the type based on the assigned value and enforces type-checking.

This is still different from dynamically typed languages like Python or JavaScript, where variable types can change at runtime.

```
var number = 42;      // Compiler infers as int
number = "Hello";     // Compilation Error: Type mismatch
```

Even with var, Java still enforces strict rules about type usage and does not allow implicit type coercion.

```
var x = 10;         // Inferred as int
var y = "Hello";    // Inferred as String

int z = x + y;      // Compilation Error: Incompatible types
```

Key Restrictions of var:

- Cannot be used without initialization: The compiler needs the right-hand side value to infer the type.
- Type is fixed after inference: Once the type is inferred, it cannot change.
- Only for local variables: var cannot be used for Class fields, Method parameters and Return types
- Not allowed with null as an initializer

```
var num;  // Compilation Error: Cannot infer type for variable

var value = 10;  // Inferred as int
value = "Hello"; // Compilation Error: Type mismatch

var value = null; // Compilation Error: Cannot infer type for variable
```

20. What are the two main categories of data types in Java?

1. Primitive Data Types

These are the basic building blocks for data manipulation. They store simple values and include:

- boolean: true/false values.
- Numeric types like byte, short, int, long, float, and double.

- char: A single Unicode character.

2. Non-Primitive (Reference) Data Types

These store references (memory addresses) rather than the actual data. Examples:

- Strings: Represent sequences of characters.
- Arrays: Collections of elements of the same type.
- Classes and Objects: Represent complex structures with attributes and methods.

21. What are the 8 primitive data types in Java, and what does each represent?

Below are the 8 primitive data types in Java.

Boolean: Represents logical values (true or false).

```
boolean isJavaFun = true;
```

Byte: 8-bit signed integer. Used for Efficient memory storage, e.g., in large arrays.

```
byte b = 100;
```

Char: Represents a single 16-bit Unicode character. Used for Characters like a, 1, or symbols.

```
char grade = 'A';
```

Short: 16-bit signed integer. Used for Saving memory for small numbers.

```
short age = 200;
```

Int: 32-bit signed integer. Used for Default for whole numbers.

```
int salary = 50000;
```

Long: 64-bit signed integer. Used for Large numeric ranges.

```
long population = 8_000_000_000L;
```

Float: 32-bit single-precision floating-point. Used For decimals when precision is not critical.

```
float pi = 3.14f;
```

Double: 64-bit double-precision floating-point. Use for Default for decimals or scientific calculations.

```
double e = 2.718281828459045;
```

22. Why does char use 2 bytes in Java, and not 1 byte like in C/C++?

Java uses the Unicode system to support characters from all languages worldwide (e.g., Latin, Greek, Chinese).

But Unicode requires 2 bytes (16 bits) to represent all possible characters.

By contrast, ASCII, used in C/C++, is limited to 1 byte (8 bits), supporting only 256 characters.

23. What is the difference between float and double in Java? When would you use one over the other?

Double is more precise and better for computations requiring high accuracy, whereas float is more memory-efficient.

- Float is used when memory is limited (e.g., in embedded systems or large datasets).
- But Double is used as a default for decimal values, scientific and financial calculations.

```
float pi = 3.14159f; // Use 'f' to indicate float
double e = 2.718281828459045;
```

24. What is widening and narrowing type conversion? Provide examples of each.

Widening Conversion is process of Implicit conversion from a smaller to a larger type. Example: int to long or float to double

```
int num = 100;
double result = num; // Implicit widening
```

Narrowing Conversion is process of Explicit conversion from a larger to a smaller type (requires casting). Example: double to int

```
double value = 9.99;
int truncated = (int) value; // Explicit narrowing
```

25. What happens if you assign a larger data type to a smaller data type?

Java will throw a compile-time error, as narrowing requires explicit casting.

```
long bigNum = 100000L;
int smallerNum = bigNum; // Compilation Error
```

26. How does Java handle type safety in mixed-type arithmetic operations?

Java automatically promotes smaller types to the larger type in mixed operations.

```
int num = 5;
float value = 2.5f;
float result = num + value; // num promoted to float
```

27. When should you use byte and short instead of int?

Use byte and short when:

- Memory is constrained.
- You're working with fixed ranges (e.g., file streams or network protocols).

```
byte age = 127; // Fits within byte range
```

28. What are wrapper classes in Java, and how do they relate to primitive types?

Wrapper classes bridge the gap between primitive types and Java's object-oriented nature. Wrapper classes in Java are object representations of the primitive data types.

- They allow primitive values to be treated as objects.
- They enable primitives to interact with collections and provide utility methods.
- Autoboxing and auto-unboxing simplify conversions between primitives and objects.
- Use cases: Collections (e.g., List<Integer> instead of List<int>) and Utility methods (e.g., Integer.parseInt()).

```
int num = 10; // Primitive type
Integer obj = Integer.valueOf(num); // Primitive to Wrapper (Boxing)

int value = obj.intValue(); // Wrapper to Primitive (Unboxing)
```

- Boxing is the process of converting a primitive type to its corresponding wrapper class.
- Unboxing is the process of converting a wrapper class back to its primitive type.
- Wrapper classes allow primitives to be stored in collections.
- Wrapper classes provide methods to perform operations on primitive values.

• Wrapper classes can represent the absence of a value using null, while primitives cannot.

```
ArrayList<Integer> list = new ArrayList<>();
list.add(10); // Autoboxing of int to Integer

String str = "123";
int num = Integer.parseInt(str); // Converts String to int

Integer obj = null; // Allowed
int num = null;     // Compilation Error
```

```
public class WrapperExample {
    public static void main(String[] args) {
        // Primitive type
        int num = 5;

        // Wrapper class (Boxing)
        Integer obj = num; // Autoboxing

        // Using utility method
        String str = obj.toString(); // Converts Integer to String

        // Unboxing
        int value = obj; // Auto-unboxing
        System.out.println("Primitive: " + value);
    }
}
```

29. What are the three types of variables in Java, and how are they different?

Local Variable

A variable declared inside the body of a method. The variable can only be used within that specific method, and it is not accessible outside it.

• Local variables must be initialized before use as they do not have default values.

• They are stored in the stack memory.

• Cannot be declared as static since static implies shared across instances, while local variables are specific to a method's execution.

```
public void display() {
    int localVar = 10;  // Local variable
    System.out.println(localVar);
}
```

Instance Variable

A variable declared inside a class but outside any method or block, and not marked as static. These variables are associated with objects, meaning each object of the class has its own copy of the instance variable.

- Instance variables have default values if not explicitly initialized.
- Stored in the heap memory.
- Their values are instance-specific and not shared among different objects.

```
public class Example {
    int instanceVar;  // Instance variable

    public void show() {
        System.out.println(instanceVar); // Default value: 0
    }
}
```

Static Variable

A variable declared with the static keyword. It is not tied to any particular instance but belongs to the class itself.

- Shared across all objects of the class. Any change to a static variable reflects in all instances of the class.
- Static variables are initialized once when the class is loaded into memory.
- Stored in the method area of memory.
- Can be accessed using the class name directly, without creating an object.

```
public class Example {
    static int staticVar = 100;  // Static variable

    public static void show() {
        System.out.println(staticVar);
    }
}

// Accessing static variable
Example.staticVar = 200; // Accessed directly using the class name
```

30. What is the difference between final, static, and static final variables?

- final: Value cannot be reassigned after initialization.
- static: Shared across all instances; belongs to the class, not any instance.
- static final: A constant value shared across all instances; effectively a class-level constant.

Final Variable

A variable declared with the final keyword is a constant, meaning its value cannot be changed once it has been initialized.

- Must be initialized either at the time of declaration or inside the constructor (for instance variables).
- If the variable holds a reference to an object, the reference cannot be changed, but the object's state can be modified.
- Used to declare constants in a program.

```
public class FinalExample {
    final int MAX_AGE = 100;  // Final variable

    public void display() {
        // MAX_AGE = 101; // Compilation Error: Cannot reassign a final variable
        System.out.println(MAX_AGE);
    }
}
```

Static Variable

A variable declared with the static keyword belongs to the class rather than any specific instance of the class.

- Shared across all instances of the class.
- Memory is allocated for static variables only once, when the class is loaded into memory.
- Can be accessed using the class name directly (e.g., ClassName.variableName).
- Commonly used to store values that are shared among all instances, like configuration values or counters.

```
public class StaticExample {
    static int counter = 0;  // Static variable

    public StaticExample() {
        counter++;  // Increments for every new instance
    }

    public static void display() {
        System.out.println("Counter: " + counter);
    }
}
```

Static Final Variable

A variable that is both static and final is a constant shared across all instances of the class. It must be initialized only once and cannot be changed.

- Combines the properties of static (class-level variable) and final (constant).
- Typically used to define constants that are shared across all instances, like PI, mathematical constants, or configuration values
- The value must be initialized at the time of declaration or inside a static block.

```
public class StaticFinalExample {
    static final double PI = 3.14159;  // Static final variable
```

```
    public static void display() {
        System.out.println("Value of PI: " + PI);
    }
}
```

31. Can you declare a variable without initializing it? And what happens if you try to access an uninitialized local variable?

Yes, for instance and static variables, as they get default values. Local variables must be explicitly initialized before use.

The compiler throws an error if you try to access an uninitialized local variable because local variables do not get default values.

32. What is Object Oriented Programming (OOPs)?

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of "objects," which can contain data in the form of fields (attributes or properties) and code in the form of procedures (methods or functions). OOP focuses on organizing software into objects that interact with each other to accomplish tasks.

OOP promotes modular, maintainable, and scalable software development by organizing code into reusable and understandable components. It emphasizes concepts such as data encapsulation, code reuse, and separation of concerns, which contribute to building robust and flexible software systems.

33. What are the 4 pillars of OOPs?

The four fundamental principles of Object-Oriented Programming (OOP) are commonly referred to as the "Four Pillars of OOP." They are:

Encapsulation

- Encapsulation refers to the bundling of data (attributes or properties) and methods (functions or procedures) that operate on the data into a single unit, called a class.
- It allows the internal state of an object to be hidden from outside interference and accessed only through well-defined interfaces (public methods).
- Encapsulation provides data protection, abstraction, and modularity, enhancing code maintainability and reusability.

Inheritance

- Inheritance is a mechanism that allows a class (subclass or derived class) to inherit properties and behavior (methods) from another class (superclass or base class).
- It promotes code reuse by enabling subclasses to inherit and extend the functionality defined in their superclass.
- Inheritance facilitates the creation of a hierarchy of classes with specialized behavior, promoting code organization and scalability.

Polymorphism

- Polymorphism enables objects to be treated as instances of their superclass, allowing them to take on multiple forms.
- It allows different objects to respond to the same message (method call) in different ways, based on their specific implementations.
- Polymorphism simplifies code by allowing methods to be written to work with objects of a superclass, without needing to know the exact subclass at compile time.
- Polymorphism is achieved through method overriding and method overloading.

Abstraction

- Abstraction involves simplifying complex systems by focusing on the essential properties while hiding unnecessary details.

- It allows developers to create abstract classes and interfaces to define common behavior without specifying implementation details.
- Abstraction helps in managing complexity, improving code maintainability, and promoting code reuse by providing a clear separation between interface and implementation.

These four principles form the foundation of Object-Oriented Programming and guide the design and implementation of software systems, promoting modular, maintainable, and scalable code.

```
class Person {
    // Private data members
    private String name;
    private int age;

    // Constructor
    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    // Public method to access private data
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        if (age > 0) {
            this.age = age;
        }
    }
}

public class EncapsulationExample {
    public static void main(String[] args) {

        //Think of a capsule as a person.
        // The person's name and age are private information,
        // and they have methods to get or update this information safely.
        Person person = new Person("John", 30);
        System.out.println("Name: " + person.getName());
        System.out.println("Age: " + person.getAge());

        person.setAge(35); // updating age
```

```
        System.out.println("Updated Age: " + person.getAge());
    }
}
```

```
//Think of an animal family.
// Dogs and cats both inherit general
// animal characteristics but also
// have their own unique behaviors.
class Animal {
    public void eat() {
        System.out.println("This animal eats food.");
    }
}

class Dog extends Animal {
    public void bark() {
        System.out.println("The dog barks.");
    }
}

public class InheritanceExample {
    public static void main(String[] args) {
        Dog dog = new Dog();
        dog.eat(); // inherited method
        dog.bark(); // specific to Dog class
    }
}
```

```
//Runtime Polymorphism
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();  // Upcasting
        animal.sound();             // Calls Dog's overridden method
    }
```

```
}
```

```
//Think of an ATM machine.
// You interact with it to withdraw money,
// check your balance, etc.,
// without needing to know how it processes
// transactions internally.
abstract class ATM {
    abstract void withdrawMoney();
    abstract void checkBalance();
}

class MyATM extends ATM {
    @Override
    void withdrawMoney() {
        System.out.println("Money withdrawn.");
    }

    @Override
    void checkBalance() {
        System.out.println("Balance checked.");
    }
}

public class AbstractionExample {
    public static void main(String[] args) {
        ATM atm = new MyATM();
        atm.withdrawMoney(); // Outputs: Money withdrawn.
        atm.checkBalance(); // Outputs: Balance checked.
    }
}
```

34. What is Compile Time Polymorphism and Runtime Polymorphism?

Compile-time polymorphism and runtime polymorphism are two forms of polymorphism in object-oriented programming. They occur at different stages of program execution and are achieved through method overloading and method overriding, respectively.

Compile-Time Polymorphism (Static Binding)

- Compile-time polymorphism occurs when the decision about which method to call is made at compile time based on the method signature.
- It is achieved through method overloading, where multiple methods with the same name but different parameter lists (number or type of parameters) are defined within the same class.
- The compiler determines the appropriate method to invoke based on the number and types of arguments provided in the method call.

• Compile-time polymorphism is also known as static binding because the method call is resolved at compile time and cannot be changed at runtime.

```
class Calculator {
    // Method overloading
    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }
}

public class Main {
    public static void main(String[] args) {
        Calculator calc = new Calculator();
        int result1 = calc.add(2, 3);          // Calls int add(int a, int b)
        double result2 = calc.add(2.5, 3.5);   // Calls double add(double a, double b)
    }
}
```

Runtime Polymorphism (Dynamic Binding)

• Runtime polymorphism occurs when the decision about which method to call is made at runtime based on the actual type of the object.

• It is achieved through method overriding, where a subclass provides a specific implementation of a method that is already defined in its superclass.

• The JVM determines the appropriate method to invoke based on the actual object type (not the reference type) at runtime.

• Runtime polymorphism allows for flexibility and extensibility in object-oriented designs, as it enables subclass objects to exhibit behavior specific to their type.

```
class Animal {
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Animal animal = new Dog();  // Upcasting
        animal.sound();             // Calls Dog's overridden method
    }
}
```

In summary, compile-time polymorphism is achieved through method overloading and resolved at compile time, while runtime polymorphism is achieved through method overriding and resolved at runtime based on the actual object type. Each form of polymorphism provides flexibility and allows for code reuse in object-oriented programming.

35. What are the different types of inheritance?

Inheritance is a fundamental concept in object-oriented programming that allows a class (subclass or derived class) to inherit properties and behavior (methods) from another class (superclass or base class). There are several types of inheritance, based on the relationship between the subclass and superclass:

Single Inheritance

- Single inheritance occurs when a subclass inherits from only one superclass.
- It is the simplest form of inheritance and is widely supported in most object-oriented programming languages.

```
class Animal {
    // Animal properties and methods
}

class Dog extends Animal {
    // Dog inherits from Animal
}
```

Multilevel Inheritance

- Multilevel inheritance occurs when a subclass inherits from another subclass, forming a chain of inheritance.
- It allows for the creation of a hierarchy of classes with multiple levels of specialization.

```
class Animal {
    // Animal properties and methods
}

class Mammal extends Animal {
    // Mammal inherits from Animal
}

class Dog extends Mammal {
    // Dog inherits from Mammal
```

```
}
```

Hierarchical Inheritance

- Hierarchical inheritance occurs when multiple subclasses inherit from the same superclass.
- It allows for the creation of a hierarchy of classes with different levels of specialization, branching out from a common superclass.

```
class Animal {
    // Animal properties and methods
}

class Dog extends Animal {
    // Dog inherits from Animal
}

class Cat extends Animal {
    // Cat inherits from Animal
}
```

Multiple Inheritance via Interfaces in Java

- Multiple inheritance occurs when a subclass inherits from multiple superclasses.
- While Java does not support multiple inheritance of classes due to the diamond problem, it supports multiple inheritance through interfaces.
- Interfaces in Java allow a class to inherit abstract methods from multiple interfaces, enabling a form of multiple inheritance.

```
interface Swimmer {
    void swim();
}

interface Flyer {
    void fly();
}

class Bird implements Swimmer, Flyer {
    // Bird implements both Swimmer and Flyer interfaces
}
```

36. What is diamond problem in Java?

The diamond problem occurs when a class inherits from two or more classes, each of which inherits from the same superclass. This creates a diamond-shaped inheritance hierarchy, where a subclass inherits the same superclass through multiple paths.

The problem arises when the subclass attempts to inherit or override a method from the common superclass. In such cases, the compiler may not be able to determine which version of the method to use, leading to ambiguity and potential conflicts.

```
    A
   / \
  B   C
   \ /
    D
```

In this diamond-shaped inheritance hierarchy, both classes B and C inherit from class A, and class D inherits from both B and C. If class A has a method foo(), and classes B and C override foo() with different implementations, the compiler may not be able to determine which version of foo() to use in class D.

In Java, the diamond problem is avoided by allowing single inheritance of classes and supporting multiple inheritance only through interfaces. Interfaces do not contain any implementation, so there is no ambiguity in method resolution.

37. What is difference between abstract class and interface?

Abstract classes and interfaces are both used to achieve abstraction and define contracts for classes in Java, but they have some key differences:

Abstract Class

- An abstract class is a class that cannot be instantiated on its own and may contain abstract methods (methods without implementation) as well as concrete methods (methods with implementation).
- Abstract classes can have constructors, instance variables, and non-abstract methods, in addition to abstract methods.
- A subclass must provide implementations for all abstract methods defined in its abstract superclass, or it must itself be declared abstract.
- Abstract classes are used to define common behavior and characteristics shared among subclasses, serving as a blueprint for creating concrete subclasses.

```
abstract class Shape {
    // Abstract method
    abstract double area();

    // Concrete method
    void display() {
        System.out.println("Displaying shape");
    }
}
```

Interface

- An interface is a reference type similar to a class but contains only abstract methods, constants (static

final fields), and default/static methods (methods with default implementations).

• Interfaces cannot have constructors or instance variables, only method signatures.

• A class implements an interface by providing concrete implementations for all the methods declared in the interface.

• Interfaces are used to define contracts that classes can choose to implement, enabling multiple inheritance of type (a class can implement multiple interfaces).

```
interface Drawable {
    // Abstract method
    void draw();
}
```

Key Differences

• An abstract class can have both abstract and concrete methods, while an interface can have abstract methods, default methods, and static methods.

• A class can extend only one abstract class, but it can implement multiple interfaces.

• Abstract classes can have constructors, whereas interfaces cannot.

• Abstract classes are used to provide a common base for subclasses to extend, while interfaces define contracts that classes can implement to achieve polymorphism.

```
// Abstract class
abstract class Animal {
    private String name;

    // Constructor in abstract class
    public Animal(String name) {
        this.name = name;
    }

    // Abstract method (no implementation)
    public abstract void makeSound();

    // Concrete method
    public void sleep() {
        System.out.println(name + " is sleeping.");
    }
}

// Interface with abstract methods and default method
interface Pet {
    // Abstract method
    void play();

    // Default method (concrete method)
    default void beFriendly() {
        System.out.println("Being friendly.");
```

```
    }

    // Static method (concrete method)
    static void showAffection() {
        System.out.println("Showing affection.");
    }
}

// Class that extends abstract class and implements interface
class Dog extends Animal implements Pet {
    public Dog(String name) {
        super(name); // Call the constructor of the abstract class
    }

    @Override
    public void makeSound() {
        System.out.println("Dog barks.");
    }

    @Override
    public void play() {
        System.out.println("Dog is playing.");
    }
}


public class AbstractVSInterfaceExample {
    public static void main(String[] args) {
        Dog myDog = new Dog("Buddy");
        myDog.makeSound(); // Dog barks.
        myDog.sleep(); // Buddy is sleeping.
        myDog.play(); // Dog is playing.
        myDog.beFriendly(); // Being friendly.

        Pet.showAffection(); // Showing affection.
    }
}
```

38. When to use abstract class VS when to use interface?

Use Interface When

A is capable of [doing this] scenario fits in your requirement. For example: below Programmable interface is capable of doing Programming

```
interface Programmable {
        void programming();
}
```

• You want to define a contract for classes to implement: Interfaces specify a set of methods that implementing classes must provide, defining a clear contract for how classes interact with each other.

• You want to achieve multiple inheritance of type: Classes can implement multiple interfaces, allowing them to inherit behavior from multiple sources without the restrictions of single inheritance.

• You want to decouple implementation from interface: Interfaces provide a way to separate the specification of behavior from its implementation, promoting loose coupling and flexibility in design.

Use Abstract Class When

• A is B or C is B scenario fits in your requirement, for example: Dog is Animal, Cat is Animal..etc.

• You want to provide a common base implementation for subclasses: Abstract classes can contain both abstract and concrete methods, allowing you to define common behavior that subclasses can inherit.

• You want to define a class hierarchy: Abstract classes are useful for creating a hierarchical structure of classes where each subclass represents a more specialized version of the superclass.

• You want to share code among closely related classes: Abstract classes can contain state and behavior that is common to multiple subclasses, reducing code duplication.

39. What is Constructor and different types of Constructors?

A constructor in Java is a special type of method that is automatically called when an instance (object) of a class is created. It is used to initialize the newly created object and set its initial state. Constructors have the same name as the class and do not have a return type, not even void.

There are several types of constructors in Java:

Default Constructor

• A default constructor is a non-parameterized constructor automatically provided by the Java compiler if no other constructor is explicitly defined in a class.

• It initializes the object with default values (e.g., numeric fields are initialized to 0, references are initialized to null).

```
public class MyClass {
    // Default constructor
    public MyClass() {
        // Constructor body
    }
}
```

Parameterized Constructor

• A parameterized constructor is a constructor with parameters that allows you to initialize the object with specific values passed as arguments.

• It enables you to customize the initialization of objects based on the values provided at the time of object creation.

```
public class MyClass {
    // Parameterized constructor
    public MyClass(int x, int y) {
        // Constructor body
    }
}
```

Copy Constructor

- A copy constructor is a constructor that initializes an object by copying the state of another object of the same class.
- It allows you to create a new object with the same state as an existing object, providing a convenient way to clone objects.

```
public class MyClass {
    private int x;

    // Copy constructor
    public MyClass(MyClass obj) {
        this.x = obj.x;  // Copy state from obj
    }
}
```

40. Is there a destructor concept in Java?

No, Java does not have a destructor concept like some other programming languages such as C++. In Java, objects are automatically garbage collected by the JVM (Java Virtual Machine) when they are no longer referenced or needed.

There was finalize() method as a workaround for the java destructor but it is also deprecated since Java 9.